Natural Language Processing

Project Ideas

General Objective

Each team will build a **complete NLP pipeline** that includes:

- Text preprocessing
- Feature extraction
- Baseline modeling
- Advanced modeling
- Evaluation and comparison
- (***Optional with bonus***) enhancement or extension

General Requirements (MANDATORY for ALL projects)

Each team must:

1. Apply **text preprocessing techniques.**
2. Use **two feature extraction methods**:
   - TF-IDF (required)
   - One embedding method (Word2Vec, GloVe, Transformer embeddings, ….)
3. Implement **at least TWO models**:
   - One **baseline model** (e.g., Logistic Regression, Naive Bayes)
   - One **advanced model** (e.g., LSTM or Transformer)
4. Perform a **clear comparison** between models:
   - Accuracy (or relevant metric)
   - Strengths & weaknesses
5. Provide **evaluation metrics**:
   - Accuracy - Precision / Recall / F1-score - Confusion Matrix
6. Submit a **final report.**

Project 1: Smart Review Analyzer

1. **Objective**
   - Build a system that analyzes user reviews.
   - And Classifies sentiment (positive / negative).
   - Extracts useful insights.

   "Extracting insights" means finding useful information from text such as ***important words***, ***common patterns***, or ***reasons*** **behind predictions**.

   Example of *"Extracting Insights"*:

   Review:

   "The service was slow"

   Basic output: (✗)

   Negative

   Improved output: (✓) → you must do like this.

   Negative
   Reason: "slow service".

   You must also extract:

   - Important keywords (e.g., *slow*).
   - Common patterns across reviews.
   - Simple statistics (e.g., % of positive vs negative reviews).

   Common patterns mean: The system should identify patterns such as:
   Most complaints are related to *delivery speed.*
   Most positive feedback is related to *food quality.*

2. **Dataset:** From your choice.

3. **Tasks**
   1. **Preprocessing**
      - Convert text to lowercase
      - Remove punctuation
      - Remove stopwords
      - Tokenization

2. **Feature Extraction**
   - TF-IDF
   - Word embeddings (Word2Vec / GloVe / BERT embeddings)

3. **Baseline Model:** Logistic Regression or Naive Bayes
4. **Advanced Model:** LSTM or Transformer (e.g., BERT) , … .
5. **Evaluation**
   - Accuracy
   - Confusion Matrix
   - Precision, Recall, F1-score
   - Compare both models

6. **Report (write what you did in details)**

   Include:
   - Description of the system
   - Dataset used
   - Preprocessing steps
   - Models used
   - Results and comparison
   - Conclusion

Project 2: Intelligent Search Engine

1. Objective

- Build a system that retrieves relevant documents based on a user query.
- Rank documents from most relevant to least relevant.
- Improve search quality using semantic understanding.

"Retrieving relevant documents" means:
Finding the most related texts to the user query based on meaning, not just exact matching words.

Example of Search:

Query: "food delivery problem"

Basic output (✗):
Returns documents that only contain the exact words "food", "delivery", "problem".

Improved output (✓) → you must do like this:
Returns documents such as:

- "The delivery was slow"
- "Late shipment issues"

Even if the exact words are not the same, the meaning is similar.

You must also implement:

- Ranking results (best results appear first)
- Top-k retrieval (e.g., top 5 results)
- Semantic similarity (understanding meaning, not just words)

2. **Dataset:** (From your choice)

Examples:

- News articles
- Wikipedia pages
- Product descriptions

3. **Tasks**

1. Preprocessing

- Convert text to lowercase
- Remove punctuation
- Remove stopwords
- Tokenization

2. Feature Extraction

- TF-IDF
- Word embeddings (Word2Vec / GloVe / BERT embeddings, …)

3. Baseline Model

- Cosine similarity using TF-IDF.

4. Advanced Model

- Semantic search using embeddings
- OR Transformer-based embeddings (e.g., BERT)

5. Evaluation

- Evaluate search quality using Top-k results (e.g., Top-5)
- Use manual relevance checking:
  - For each query, check how many returned documents are actually relevant.
- Calculate **Precision@k**:
  - Precision@k = (Number of relevant documents in Top-k) / k

  Example:

  Query: "food delivery problem"

  Top 5 results: ….. .

  3 relevant → Precision@5 = 3/5
- Compare baseline vs advanced model:
  - Which model retrieves more relevant results?
  - Which model understands meaning better?
- Explain your results clearly

in notebook

6. Report (write what you did in detail)

Include:
- Description of the system & Dataset used
- Preprocessing steps
- Models used
- Results and comparison
- Conclusion

Project 3: Fake News Detection

1. Objective

- Build a system that classifies news articles as Fake or Real.
- Understand patterns in fake vs real news.
- Explain why a news article is classified as fake or real.

"Explaining predictions" means:
Finding reasons behind the model decision such as important words or patterns that led to classification.

Example of Classification:

News: "Breaking: You won't believe what happened next!"

Basic output (✗):
Fake

Improved output (✓) → you must do like this:
Fake
Reason: "you won't believe", "breaking"

<u>You must extract:</u>

- Important keywords (e.g., shocking, breaking)
- Common patterns across fake news
- Simple statistics (e.g., % of fake vs real news)

Common patterns mean:
The system should identify patterns such as:

- Fake news often uses exaggerated words (e.g., "shocking", "unbelievable")
- Real news uses more formal and factual language

2. Dataset: From your choice.

Examples:

- Fake News datasets (Kaggle)

3. Tasks

1. Preprocessing

- Convert text to lowercase
- Remove punctuation
- Remove stopwords
- Tokenization

2. Feature Extraction

- TF-IDF
- Word embeddings (Word2Vec / GloVe / BERT embeddings)

3. Baseline Model

- Logistic Regression

4. Advanced Model

- LSTM or Transformer (e.g., BERT)

5. Evaluation

- Accuracy - Confusion Matrix - Precision, Recall, F1-score
- Compare both models

6. Report (write what you did in detail)

Include:
- Problem description & Dataset description
- Preprocessing steps
- Models used
- Results and comparison
- Conclusion

Project 4: Text Summarization System

1. Objective

Build a system that summarizes long text into a shorter version.
- Keep the most important information.
- Reduce text length without losing meaning.

"Summarization" means:
Generating a shorter version of a text while preserving its main ideas and important information.

Example of Summarization:

Text:

"The food was amazing and the service was excellent. The restaurant was clean and the staff were friendly. However, the delivery was very slow and the order arrived late."

Basic output (✗): Returns random or incomplete sentences

The restaurant was clean and the staff were friendly

Improved output (✓) → you must do like this:

The food and service were good, but the delivery was slow.

You must also ensure:

- The summary is shorter than the original text
- The main idea is preserved & Important information is included

Important sentences are selected based on their score (using TF-IDF) and their meaning (using embeddings), where higher-score or more representative sentences are chosen for the summary.

What does "important information" mean?!

The system should identify:

- Key sentences
- Main ideas
- Important keywords

2. Dataset: From your choice

Examples:

- News articles
- Blog posts
- Reviews datasets

3. Tasks

1. Preprocessing

- Convert text to lowercase
- Remove punctuation
- Remove stopwords
- Tokenization

2. Feature Extraction

- TF-IDF
- Word embeddings (e.g. Word2Vec / GloVe / BERT embeddings)

3. Baseline Method

- Extractive summarization using TF-IDF
(e.g., selecting top important sentences)

4. Advanced Method

▪ Transformer-based summarization
(e.g., using pre-trained models like BERT or similar)

5. Evaluation

▪ Compare original text vs summary
▪ Check if key information is preserved
▪ Compare baseline vs advanced method
▪ Manual evaluation (quality of summary)

6. Report (write what you did in detail)

Include:
▪ Problem description
▪ Dataset used
▪ Preprocessing steps
▪ Methods used
▪ Results and comparison
▪ Conclusion

Project 5: Keyword Extraction System

1. Objective

▪ Build a system that extracts the most important keywords from a text.
▪ Identify the main ideas without reading the full text.
▪ Highlight key information automatically.

“Keyword extraction” means:
Finding the most important words or phrases that represent the main idea of the text.

Note: Keywords can be single words or phrases (e.g., "machine learning")

Example:

Text:

"Artificial Intelligence is transforming healthcare by improving diagnosis and patient care using advanced machine learning models."

Basic output (✗):

"is, the, by, using"

Improved output (✓) → you must do like this:

"Artificial Intelligence", "healthcare", "diagnosis", "machine learning"

You must also extract:

- Important concepts
- Repeated meaningful terms

What makes a keyword "important"?

The system should consider:

- Words that appear frequently in the text
- Words that are not common (not stopwords)
- Words related to the main topic

2. Dataset: From your choice

Examples:

- Articles
- News
- Scientific abstracts

3. Tasks

1. Preprocessing

- Convert text to lowercase
- Remove punctuation
- Remove stopwords
- Tokenization

2. Feature Extraction

- TF-IDF
- Word embeddings

3. Baseline Method

- TF-IDF-based keyword extraction (select top important words)

4. Advanced Method

- Embedding-based extraction
(e.g., similarity or clustering)

5. Evaluation

- Check if extracted keywords represent the text
- Compare baseline vs advanced
- Manual evaluation

6. Report (write what you did in detail)

Include:
- Problem description
- Dataset used
- Preprocessing
- Methods used
- Results
- Comparison
- Conclusion